Formal Verification Project Report

Team details

Keith Begley 10261631
Ciarán Cope 21363716

Brief description of collaboration

We collaborated both in the initial parts of the project in preparing the methods, and also in the final part to verify them. In both cases we each attempted construction of a method or its verification individually and then reviewed afterwards, coming to an agreement about what the final method should be like. We employed a repository on Github for the sharing of code and regularly pushed code to our own separate branches before discussing and merging.

Use of AI tools

For proving the methods in the final part of the project, we used VSCode with Github Copilot in Agent mode using the models GPT 5.2 and GPT 5.1-Codex-Max**.** GPT 5.2 was used for all proofs except two. addtoSet() was proved by Dafny without assistance. setProduct() required us to switch to another model GPT 5.1-Codex-Max. This is described later in this report.

We had a couple of failed attempts to make progress with proofs over several hours without using AI, but eventually gave in and let AI do most of the heavy lifting. So, this very much became an experiment in how effective generative AI is at preparing code for formal verification. We initially attempted the whole file at once, but it seemed to work better, and was less unwieldy, when a new chat session was begun for each method and the focus was put on invariants for loops. Prompts like 'Find the while loop invariants in checkSet and verify' were used for each method in turn. For 4 out of the 9 methods this procedure got the method to verify in one shot. One method, addtoSet(), was already proved by Dafny without assistance.

Often there was more included in the proofs than was necessary. This happened especially when there was a boolean guard on while loops. There would be a further *if b* {...} block after the main code. For example:

```
// Prove b ==> isEvenSet(s)
 if b {
    // loop can only stop with b==true if i reached the end
    assert i == |s|;
    assert isSet(s); // from b ==> bs and bs ==> isSet(s)
    // Turn the index-based invariant into the membership-based
predicate
    assert forall x :: x in s ==> isEven(x) by {
      forall x | x in s
        ensures isEven(x)
      {
        ghost var p: int :| 0 <= p < |s| && s[p] == x;
```

```
        assert p < i;                // since i == |s|
        assert isEven(s[p]);         // from the loop invariant
      }
    }
    assert isEvenSet(s);
  }
```

Dafny did not need this to verify the method. Sometimes these blocks were quite long. We removed all unnecessary statements in the final version. Sometimes there were also single or multiple asserts from blocks that could be removed without harm to the verification of the methods.

Sometimes the AI would also struggle with the syntax. This happened especially with *assume* statements. When these errors were pointed out it was often not able to correct them without producing further errors. Other common problems included unresolved identifiers, ghost variables outside of specification contexts, and putting semi-colons after closing braces.

Sometimes the AI failed to get the method to verify, but Dafny would helpfully mark a particular postcondition or invariant in need of attention. For example, this happened when working on the intersection() method. The model was prompted to address the error with: 'Try again paying particular attention to the postcondition: ensures forall x :: (0 <= x < |t| ==> t[x] in s1 && t[x] in s2)' and was able to solve the issue. This suggests that it would be possible for a similar workflow to be automated with errors returned directly from Dafny to the AI. However, this did not always work (see our description of setProduct() later in this report).

Code modifications

checkEvenSet()

Original code

```
method checkEvenSet(s: seq<int>) returns (b: bool){
  b := true; // Initialise at the start
  var i := 0;
  while i < |s|
    {
      if s[i] % 2 != 0 {
        b := false;
      }
      i := i + 1;
    }
}
```

Modified code

```
method checkEvenSet(s: seq<int>) returns (b: bool)
  ensures b <==> isEvenSet(s)
{
  // First, decide set-ness
  var setness := checkSet(s);          // bs <==> isSet(s)
```

```
  b := setness;
  var i := 0;
  while i < |s| && b
    invariant 0 <= i <= |s|
    // If we are still returning true, then checkSet succeeded
    invariant b ==> setness
    // If we are still returning true, all processed elements are even
(in the isEven sense)
    invariant b ==> (forall k :: 0 <= k < i ==> isEven(s[k]))
    // If s is an even-set, then b can never become false
    invariant isEvenSet(s) ==> b
    decreases |s| - i
  {
    if s[i] % 2 != 0 {
      // If s is truly an even-set, this branch is impossible
      if isEvenSet(s) {
        assert s[i] in s;
      }
      b := false;
    } else {
      ModuloEven(s[i]);        // s[i] % 2 == 0 ==> isEven(s[i])
    }
    i := i + 1;
  }
}
```

The AI frontloaded checkSet() to satisfy the isSet(s) conjunct of the isEvenSet(s) predicate. In this case a special var was needed for it because a conditional *invariant b ==> setness* was needed.

It also included b as a guard for the loop. While not strictly necessary for the task of outputting a boolean, it is more efficient. This style also seems to have advantages for the verification in that conditionals having a boolean guard as an antecedent can be used as invariants, especially when the postcondition takes a similar form.

It was also necessary for it to include an else condition in order to use the lemma *ModuloEven(s[i]);*. This is a salient difference in necessary code structure for verification. Alternative branches must sometimes be made explicit.

checkOddSet()
Original code

```
method checkOddSet(s: seq<int>) returns (b: bool)
  requires isSet(s)
  ensures b <==> isSet(s)
  ensures b <==> isOddSet(s)
{
```

```
  b := true; // Initialise at the start
  var i := 0;
  while i < |s|
      invariant 0 <= i <= |s|
      decreases |s| - i
      invariant b ==> (forall v :: 0 <= v < i ==> (s[v] % 2 != 0))
    {
      if s[i] % 2 == 0 {
        b := false;
      }
      i := i + 1;
    }
    assert i == |s|;
}
```

Modified code

```
method checkOddSet(s: seq<int>) returns (b: bool)
  ensures b <==> isOddSet(s)
{
  // First, decide set-ness
  var set_ness := checkSet(s);   // set_ness <==> isSet(s)
  b := set_ness;
  var i := 0;
  while i < |s| && b
    invariant 0 <= i <= |s|
    // If we are still returning true, then checkSet succeeded
    invariant b ==> set_ness
    // If we are still returning true, all processed elements are odd
(in the isOdd sense)
    invariant b ==> (forall k :: 0 <= k < i ==> isOdd(s[k]))
    // If s is an odd-set, then b can never become false
    invariant isOddSet(s) ==> b
    decreases |s| - i
  {
    if s[i] % 2 == 0 {
      // Show this branch is impossible if s is truly an odd-set
      if isOddSet(s) {
        assert s[i] in s;
      }
      b := false;
    } else {
      ModuloOdd(s[i]);
    }
```

```
    i := i + 1;
  }
}
```

Again, the AI frontloaded checkSet(), used a boolean guard for the loop, and an else condition, similarly to the previous method.

setProduct()

Original code

```
method setProduct(s1: seq<int>, s2: seq<int>) returns (t: seq<int>)
  requires |s1| == |s2|
  requires isSet(s1)
  requires isSet(s2)
  ensures forall x: int :: 0 <= x < |t| ==> exists y: int, z: int :: 0
<= y < |s1| && 0 <= z < |s2| && t[x] == s1[y] * s2[z]
{
  var i := 0;
  var j := 0;
  t := [];
  while i < |s1|
    {
      while j < |s2| {
        var placeholder := s1[i] * s2[j];
        t := addToSet(t, placeholder);
        j := j + 1;
      }
      i := i + 1;
    }
}
```

Modified code

```
method setProduct(s1: seq<int>, s2: seq<int>) returns (t: seq<int>)
  requires isSet(s1) && isSet(s2)
  ensures isSet(t)
  ensures forall x :: x in t ==> exists i1, j1 :: 0 <= i1 < |s1| && 0
<= j1 < |s2| && x == s1[i1] * s2[j1]
  ensures forall i1, j1 :: i1 in s1 && j1 in s2 ==> i1 * j1 in t
  ensures isEvenSet(s1) || isEvenSet(s2) ==> isEvenSet(t)
  ensures isOddSet(s1) && isOddSet(s2) ==> isOddSet(t)
{
  var i := 0;
  t := [];

  while i < |s1|
```

```
    {
    var j := 0;
```

The code snippet is truncated for brevity. Here there was an error in our original code, which the AI spotted. The variable *j* should of course be reset to 0 for each *i*.

invertParitySet()
Modified & Original code

```
  {
    // var t0 := t;
    // var i0 := i;
    // var v := invertParity(s[i]);
    // t := addToSet(t0, v);
    // i := i0 + 1;
    t := addToSet(t, invertParity(s[i]));
    i := i + 1;
  }
```

Here, in the while loop, the AI made a peculiar shuffle of variables for no apparent purpose. We have reinstated our original code.

Reflection

Which methods were hardest and why

The method that we expected would be most difficult for Dafny to verify was the invertParitySet(). There are a few reasons for this. Firstly, there is a mismatch between sequences and sets. T and S are declared as sequences at the beginning of the method, but within the verification we specify them as sets to satisfy the proof. This means that the verifier has to constantly bridge the gap between the following quoted invariant and assert.

```
invariant forall k :: 0 <= k < i ==> t[k] == invertParity(s[k])
```

```
assert forall x :: x in t ==> isOdd(x)
```

This requires nontrivial reasoning surrounding membership versus indexing, which Dafny does not infer automatically. Moreover, the verification relies on existential quantification to relate the set members, to the sequence indices as,

```
ghost var p: int :| 0 <= p < |t| && t[p] == x;
```

Existential values are handled conservatively by the verifier, and reasoning surrounding their properties requires further manual assertions. This complicates the proof and weakens the automation for verification.

Despite this expectation the problems that actually afflicted AI attempts to prepare the code for verification included unresolved identifiers and ghost variables outside specification contexts. It also

spotted that its own attempts included redundant invariants and offered to remove them. As noted above, it also made extraneous edits to the original code.

The hardest method to get Dafny to verify was the *setProduct()* method, as it caused the most issues for the AI. There was an error in our original code, which probably did not help matters, but it was nevertheless corrected on every attempt. After working through some syntax errors in a very long proof, including ghost variables outside specification contexts, and drawing attention to a troublesome invariant that remained unproved, Dafny looked as if it might verify the output. However, it was hitting a timeout of 20 seconds. We duly increased the timeout, but the verification continued indefinitely until we received warnings from the OS that the hard drive was being filled up (about 40GB). The machine was hastily restarted to resolve the issue! When queried, the AI reported that it was likely that "Z3 is drowning in **nested quantifiers**". After some further failed attempts to get GPT 5.2 to solve this issue, we decided to switch the model to GPT 5.1-Codex-Max. This immediately produced a shorter proof in one shot that was verified.

How proof attempts changed our understanding of specification and correctness

The proof attempts did not fundamentally change our understanding of the intended behavior of the methods, as the implementations were already largely correct. Instead, the verification process helped to refine and clarify details that were implicit or only informally justified. In this sense, the method was approximately correct from the outset, and the remaining effort involved aligning the invariants and conditions more precisely with what was already known to be true. There were also some realisations that intuitive correctness, or apparent simplicity, may not translate to a formal verification process. Sometimes solutions can be written that might make intuitive sense but result in Dafny being incapable of verifying without a complex proof. Also, sometimes verification processes, as computational processes themselves, are apparently non-terminating.

Additionally, the difficulty of maintaining loop invariants demonstrated how important invariants are as a bridge to the specification of a program. The need to strengthen invariants properly in order to satisfy the postcondition emphasised that correctness is not only a property checked at the end of execution, but something that must be preserved throughout. Minor improvements of loop invariants and assertions did not change the overall structure of the method, but ensured that the final postcondition could be proved and that correctness could be held at each iteration.

Whether AI helped or slowed us down

AI definitely helped for the most part, especially at the beginning. It took a significant amount of time to get off the ground before using AI, with numerous failed attempts and quite a few struggles with the verifications. However, once AI was introduced, the process significantly improved as it could get the solution for some of the methods in one shot. As alluded to previously, even in scenarios where AI did not solve the entire problem, it could get it to a point where we could intuitively focus on fixes and workarounds, including opting for the output of different models.

As well as this, in cases where some of our code needed to be modified or was even incorrect, we may not have realised this so quickly. We may have even continued to struggle to verify incorrect code before eventually realising the problem, it at all. If the method made intuitive sense in our heads, then we would have been satisfied with it and assumed that the problems were with the invariants or assertions, so AI was quite a large help in this scenario.

Other thoughts on the verification experience

The project was certainly made a lot easier by using AI. However, there is a concern that we did not gain much more practice of doing verification ourselves. We have certainly learnt some interesting facts about verification from watching the AI do it, or seeing its complete outputs, but we are perhaps no more prepared for a paper and pencil examination on it as we worked through very little of it ourselves. As it happens, this kind of phenomenon is the very subject of one of our team member's current doctoral research. So, this apparent drawback has nevertheless acted as a valuable learning experience in quite another way.